Helix Robotics ? Platform Architecture Overview

Last updated: 2026-03-12 by Yui Hayashi
Status: Living document ? services churn; if you spot drift, ping ‘#platform-eng‘ in Slack.

Tier-0 services (customer-facing)

These are the services whose downtime is visible to customers.

identity-svc

Owner: **Marcus Chen** (identity@helix.internal)
Repo: ‘github.com/helix-robotics/identity-svc‘
Hostname: ‘auth.helix.io‘ (customer-facing) and ‘identity.helix.internal‘ (intra-cluster). Runs on EKS in ‘us-west-2‘, 6 replicas behind an ALB.

Stack: Go 1.22, PostgreSQL (RDS), Redis (ElastiCache). Provides OAuth2 Authorization Code + PKCE for customer portals, OIDC discovery, and SCIM 2.0 for customer-managed users. Customer JWTs are HS256-signed with secrets rotated quarterly via Vault.

Critical paths:

- Token issuance (‘POST /oauth/token‘) ? p99 < 80ms.
- JWKS endpoint (‘/.well-known/jwks.json‘) ? cached 4 hours.
- SCIM endpoints ? Stripe-style cursor pagination.

payments-api

Owner: **Sam Liu** (sam.liu@helixrobotics.com)
Repo: ‘github.com/helix-robotics/payments-api‘
Hostname: ‘payments.helix.io‘ and ‘payments.helix.internal‘.

Stack: Python 3.11, FastAPI, PostgreSQL (RDS). Charges flow through Stripe; we hold no card data ourselves ? PCI scope is SAQ-A. Vault holds Stripe restricted keys per environment.

Note: ‘payments-api‘ writes to ‘pii-vault‘ (internal) for any billing-address PII we receive from OEMs.

dashboard-web

Owner: **Lukas Bauer** (lukas.bauer@helixrobotics.com)
Repo: ‘github.com/helix-robotics/dashboard-web‘
Hostname: ‘app.helix.io‘.

Next.js 14 (App Router), deployed to Vercel for edge rendering + a small Node backend on EKS for any server-only logic. Authenticates end-users via identity-svc.

Tier-1 services (internal customer-impacting)

orders-api

Owner: Yui Hayashi (yui@helix.internal)
Hostname: ‘orders.helix.internal‘ (no external surface).

Stack: Python 3.11, FastAPI, PostgreSQL.

Processes OEM order intake. Sales reps and customer-success interact via dashboard-web, which proxies through identity-svc into orders-api.

webhook-router

Owner: Yui Hayashi
Hostname: ‘webhooks.helix.internal‘ (egress only).
Stack: Go 1.22, DynamoDB.

Receives internal events (‘charge.succeeded‘, ‘device.registered‘, etc.) from a Kinesis stream and fans out HTTPS POSTs to customer-supplied webhook URLs with HMAC-SHA256 signatures. Failures retry with exponential backoff up to 24 hours.

Outbound IPs egress through a NAT in ‘10.50.0.0/16‘ (prod VPC). We publish the NAT IP block to customers so they can allowlist.

device-registry

Owner: Yui Hayashi
Hostname: ‘devices.helix.internal‘.
Stack: Python 3.11, DynamoDB (single-table design).

Registers and authenticates physical robotics devices. Each device gets a unique X.509 client certificate signed by our internal CA. Cert rotation is automated quarterly ? see ‘runbooks/rotate-customer-api-key.md‘ (TODO: needs update for device certs specifically).

Tier-2 services (data + infra)

pii-vault

Owner: **Security team** (you, going forward)
Hostname: ‘pii.helix.internal‘.
Stack: Python 3.11, PostgreSQL on a separate RDS instance with customer-managed-key encryption, network ACL restricting ingress to the prod VPC.

Holds billing-address PII separated from the main payments DB. Access is via a narrow set of allowlisted service principals ? payments-api, dashboard-web (for redaction-aware display), orders-api.

analytics-pipeline (Snowflake)

Vendor: Snowflake Inc.
Owner: Data team (Snowflake account ‘cortex-eq39172‘, lead vendor contact via Lukas).

dbt models materialize from RDS read-replicas ? S3 ? Snowflake. PII is masked at the dbt layer (in theory; see open issue HELIX-2031).

Cross-cutting

Internal CA

A small Go service runs the internal X.509 CA used for device certificates. Currently owned by Yui; you should inherit this in month 2.

Secrets management

HashiCorp Vault, self-hosted on EKS, 3-node cluster, KV v2 engine. Auth via Kubernetes service accounts for workloads; humans use Vault OIDC backed by Okta. Sealed/unsealed via shamir keys held by Tom, Priya, Alice, and Diana.

Observability

Datadog APM + logs + metrics. Sentry for frontend errors. PagerDuty on top, three rotations:

- Platform on-call (covers identity-svc, orders-api, webhook-router, device-registry, internal CA, EKS).
- Payments on-call (covers payments-api, pii-vault).
- Frontend on-call (covers dashboard-web).

There is no security on-call yet. Diana and you are exploring whether to add one.

Known gaps

(Things written down so we don’t forget them ? not a prioritized list.)

- No bot-protection layer in front of dashboard-web. Sees periodic credential-stuffing bursts (Diana has dashboards).
- IAM has long-lived access keys for two legacy CI integrations. Tracked in HELIX-1879 (Raj’s queue).
- ‘pii-vault‘ access logs are kept but never alerted on.
- ‘webhook-router‘ lacks a postmortem and a runbook ? historically it just hasn’t broken.
- Snyk runs on PRs but the security team has no triage queue for findings (this is partly why you’re being hired).